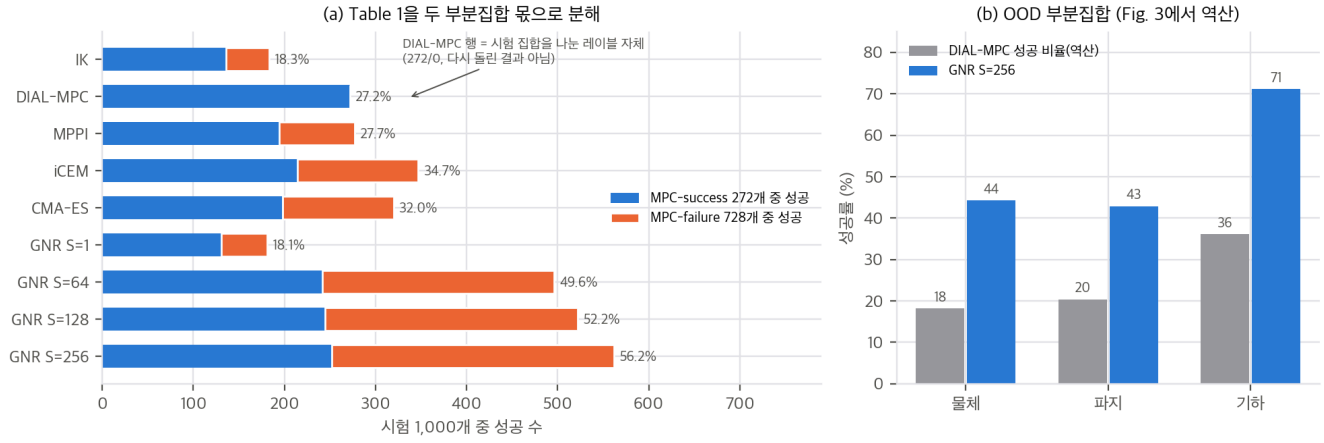
GNR: 분석 리포트

게시용 버전: 논문 그림은 라이선스(CC BY-NC-ND 4.0) 때문에 제외했습니다. 원문 https://arxiv.org/abs/2610.12440 · arXiv:2610.12440v1 · Meta Reality Labs Research (Gao, Yang, Abbatematteo, Godwin, Wang, Boldu, Man, Dou, Qin, Nakagome, Whitmire) · 공식 코드·데이터 미공개(2026-11-03 확인, YY-GX/GNR은 프로젝트 페이지 저장소) · 분석일 2026-11-03 · 논문 그림은 개인 학습용으로만 사용(논문 라이선스 CC BY-NC-ND 4.0)

핵심 요약

1. **무엇을 하는가:** sampling MPC(DIAL-MPC)가 푼 리타겟팅 궤적으로, IK 기준 제어에 더할 잔차의 조건부 flow matching 모델을 학습한다. 새 시연에서는 후보 S개를 뽑아 시뮬레이터로 거르거나(pass@S), MPC 초기값으로 쓴다(GNR+MPC). HOT3D에서 223k 궤적과 접촉력 라벨을 만들었다.
2. **"56.2% vs 27.2%"의 27.2%는 시험 집합의 분할 비율이다:** 1,000개를 DIAL-MPC 성공 272개와 실패 728개로 골랐으므로 DIAL-MPC 행은 정의상 100%/0%다. Table 1은 정수 개수로 정확히 맞는다. 다시 돌린 기준선 중 최고는 iCEM 34.7%다.
3. **이득은 거부 표집에서 나오고 꼬리가 무겁다:** GNR S=1은 IK와 같다(18.1% 대 18.3%). MPC-failure의 pass@S를 Beta-혼합으로 맞추면 꼬리 지수 a ≈ 0.12-0.14라서, 표본을 두 배로 늘려도 실패가 8-9%만 준다. ShapeFilter는 a ≈ 0.8로 표본 몇 개면 된다.
4. **"시연" 수는 최적화 시도 단위다:** 기하 변형 쪽은 쌍 7,280개 × 시드 24개다. 쌍 단위 MPC 성공률 37.4%가 시도 단위로는 27.6%가 되고, MPC-failure의 13.8%는 다른 시드로 이미 푼 쌍이다. 궤적 수는 +133%지만 서로 다른 기하는 +19.7%다.
5. **비용 수치는 출처가 섞였지만 결론은 유지된다:** 7.6배(Table 1 단가)는 Table 14 단가로 9.9배다. 2.62배 시나리오는 비용의 78%가 MPC 시드라서 최대 3.36배까지만 줄일 수 있다. ShapeFilter "CPU 비용 1.6%"는 조기 종료할 때만 성립한다(64개를 다 쓰면 약 21%). NIST, 다중 embodiment, BC의 "비슷하거나 나은 SR"은 통계적으로 구분되지 않는다.
6. **구현 가능성:** 코드·가중치·데이터·데이터 엔진이 비공개라 논문 결과는 재현할 수 없다. SuperDex(Apache-2.0, FR3 + DG-5F 모델, macOS ARM 지원 표기)와 DIAL-MPC 원 구현, HOT3D는 공개되어 작은 규모 재구현은 가능하다. 이 맥에서는 재계산과 toy를 끝냈고, SuperDex·torch(MPS) 설치 승인 시 축소판을 시도할 수 있다.



출처: Table 1, Fig. 3, 부록 D.3. 부분집합별 MPC 성공 수 37·69·166의 합이 정확히 272. analysis/table_check.py

그림: Table 1을 MPC-success 272개와 MPC-failure 728개 안의 성공 수로 분해(직접 계산, `03` §1).

01. 논문 분석: GNR (arXiv:2610.12440)

- 제목: *Generative Neural Retargeting for Human-to-Robot Dexterous Manipulation*
- 저자: Dechen Gao, Yue Yang(공동 1저자), Ben Abbatematteo, Nathan Godwin, Pengcheng Wang, Roger Boldu, Steven Man, Zhiyang Dou, Chuan Qin, Sho Nakagome, Eric Whitmire. 소속은 Meta Reality Labs Research이고, 일부 저자는 UC Davis, UNC Chapel Hill, UC Berkeley, MIT, CMU 소속입니다(Meta 재직 중 수행).
- 공개: arXiv **v1 2026-10-08**(분석 대상, 40쪽: 본문 10쪽, 참고문헌, 부록 A-J). 프로젝트 페이지 https://yy-gx.github.io/GNR/
- 코드: **공개되지 않았습니다**(2026-11-03 확인). `YY-GX/GNR` 은 프로젝트 페이지 저장소이고, README에 "Code and Dataset buttons are omitted until release"라고 적혀 있습니다. 223k 데이터셋도 아직 공개되지 않았습니다.
- 라이선스: arXiv abs 페이지 기준 **CC BY-NC-ND 4.0**. PDF 안에는 저작권 문구나 별도 권리 표기가 없습니다(`©`, copyright, rights, license 검색 0건, PDF 메타데이터 Creator는 arXiv GenPDF). CC BY가 아니므로 논문 그림은 개인용 리포트와 PPT에만 넣고, 게시본에서는 뺐습니다.
- 분석 범위: 본문과 부록 전부, 프로젝트 페이지(2026-11-03 저장)

---

1. Abstract (한 줄 요약과 주장)

한 줄 요약: 사람 시연을 로봇손 궤적으로 바꿀 때 시연마다 sampling MPC를 새로 돌리지 않습니다. 대신 MPC가 이미 푼 궤적으로 **IK 기준 제어에 더할 잔차의 조건부 생성 모델(flow matching)**을 학습하고, 새 시연에서는 후보 S개를 뽑아 시뮬레이터로 걸러 냅니다(pass@S). 정밀하고 긴 과제에서는 생성 결과를 MPC 초기값으로 씁니다.

저자들의 가설은 "동역학적으로 가능한 궤적은 시연들이 공유하는 저차원 manifold 근처에 모여 있다"입니다(Abstract, Sec. 3.1). 기여로 내세우는 것은 두 가지입니다(Sec. 1).

1. **리타겟팅을 생성 모델링으로 바꿈.** HOT3D OOD 시험 1,000개에서 GNR(S=256)은 56.2%, DIAL-MPC는 27.2%이고, 표본은 MPC의 8.5%(256/3,000), 궤적당 CPU 비용은 7.6배 적다고 주장합니다. MPC가 실패한 시연의 42.6%를 풉니다(Table 1).
2. **real-to-sim 데이터 엔진.** HOT3D에서 223k 궤적, 3.3k 물체 기하, 시뮬레이션 접촉력 라벨을 만들었습니다. motion capture로 모은 정밀 과제 두 개(ShapeFilter 1.96 mm, NIST 1 mm·29초)에서 3.4k 궤적을 만들었습니다.

---

2. Method

2.1 문제 설정 (Sec. 3.1)

리타겟팅은 IK로 얻은 운동학 레퍼런스 $\bar{x}_{0:T}$(로봇·물체 상태)를 받아, 물리에서 이를 재현하는 제어열 $u_{0:T-1}$을 찾는 제약 최적화입니다. 정식화는 SPIDER(Pan et al., 2025)를 따릅니다.

$$\min_u J(u_{0:T-1}) \text{ s.t. } x_{t+1} = f(x_t, u_t),\ x_{0:T} \in X,\ u_{0:T-1} \in U$$

접촉이 많아 J는 비볼록하고 불연속입니다. 그래서 MPPI, DIAL-MPC 같은 sampling 최적화가 표준인데, 저자들이 지적하는 문제는 두 가지입니다. 과제와 무관한 잡음으로 탐색해서 표본 효율이 낮고, **시연마다 독립적으로 풀어서 앞 시연의 해가 다음 시연에 도움이 되지 않습니다**.

2.2 생성 모델: 잔차 flow matching (Sec. 3.1, App. B.1)

- **무엇을 생성하나:** IK 기준 제어 $\bar{u}$에 대한 잔차 $\Delta = u - \bar{u}$입니다. 조건은 $c = (\bar{x}_{0:T}, o)$이고, o는 물체 기하(HOT3D: Point Transformer V3, 점 1,024개) 또는 물체 자세(ShapeFilter 본 실험, NIST)입니다.
- **학습:** 표준 conditional flow matching입니다. $z_\tau = (1-\tau)z_0 + \tau\Delta$로 두고 $\| v_\theta(z_\tau, \tau, c) - (\Delta - z_0) \|^2$를 줄입니다. 학습 쌍 (Δ, c)는 DIAL-MPC가 성공한 궤적입니다.
- **추론:** Euler 10단계로 $z_0 \to \hat{\Delta}$를 적분하고 $u = \bar{u} + \hat{\Delta}$로 씁니다.
- **구조:** temporal U-Net(폭 128·256·512) + FiLM 조건입니다. HOT3D 모델은 19.57M 파라미터이고, A100 한 장에서 28,480 에피소드를 bfloat16, 배치 128로 학습했습니다. 에폭과 학습률은 HOT3D에 대해서는 적혀 있지 않습니다. ShapeFilter와 NIST는 AdamW, 학습률

4×10^{-4}, EMA 0.999로 100·200 에폭을 학습했습니다.
- 잔차 예측이 절대 제어 예측보다 낫다는 근거는 App. E.4입니다(§3.5).

2.3 두 가지 배치 방식 (Sec. 3.2)

1. **Open-loop GNR:** 같은 조건에서 잠재 변수 S개를 뽑아 후보 궤적 S개를 만들고, 각각 시뮬레이터에서 재생해 성공 기준(Table 4)으로 판정합니다. 하나라도 통과하면 성공이고, 이를 **pass@S**로 보고합니다. 결국 "학습된 제안 분포 + 시뮬레이터 거부 표집"입니다.
2. **GNR+MPC:** 생성한 궤적을 MPC 초기값으로 주고 MPC가 다듬습니다. 오차가 누적되는 긴 과제(NIST)를 위한 방식입니다.

2.4 데이터 엔진 (Sec. 4, Fig. 2, App. C)

1. **real-to-sim:** 손 motion capture나 egocentric 복원(HOT3D)을 시뮬레이션 사람손(33-DoF)으로 옮기고, 손목 자세와 손끝 5개를 목표로 한 keypoint IK로 Tesollo DG-5F(26-D 관절 제어)에 리타겟합니다. 왼손 시연은 장면 전체를 시상면으로 뒤집어 오른손 모델로 처리합니다(App. C.1).
2. **분할:** HOT3D는 손가락 두 개 이상이 물체 표면 20 mm 안에 있는 연속 구간으로 자릅니다. 약 5,000개 구간이 나옵니다(App. C.2).
3. **파지 재설정:** 이미 쥔 상태로 시작하는 구간은 IK가 손바닥을 물체에 박아 넣을 수 있습니다. 그래서 물체를 고정하고 중력을 끈 채 손을 닫은 뒤, 약 12프레임 후 고정을 풉니다(App. C.3).
4. **DIAL-MPC 최적화와 성공 판정 Γ:** $\Gamma = \gamma_{track} \wedge \gamma_{pen} \wedge \gamma_{grasp} \wedge \gamma_{force}$. HOT3D 기준은 물체 위치 오차 150 mm·회전 40° 미만, 평균 캡슐 침투 5 mm 이하, 놓은 뒤 쥐는 힘 중앙값 1 N 이상, 손가락별 힘 95분위 100 N 이하입니다(Table 4, App. D.1).
5. **접촉력 라벨:** SDF penalty 접촉으로 접촉점마다 위치, 법선, 침투 깊이, 힘을 기록합니다. 접촉 강성은 시뮬레이터 기본값의 1/10이고, 짧은 median filter로 이상치를 지웁니다(App. C.4). 라벨 값은 이 강성 선택에 따라 달라집니다.
6. **두 단계 확장(Sec. 4.2):**
   - *발견 단계:* 원래 조건과 단일 축 섭동 16개(위치 ±15 mm, 시작 시점 ±3·±5프레임, 크기 ×1.15·×1.30, 회전 ±10°, 마찰 ×0.5·×2)로 리타겟이 되는 구간을 찾습니다.
   - *확장 단계:* 성공한 구간의 물리(마찰, LLM이 추정한 밀도 범위로 정한 질량)와 기하(SAM 3D 재생성·가림 보완·회전 후 재생성, 변형, 늘이기; 물체당 200 변형)를 바꿔 다시 최적화합니다. 변형마다 **최적화 시드 24개**를 돌립니다(App. C.5).

시뮬레이터는 SuperDex Physics(Meta의 "Mochi team" 암시적 접촉 엔진)입니다. 비볼록 충돌 형상과 공간에 분포된 3D 접촉력을 지원합니다(Sec. 5.1).

---

3. Experiments

3.1 설정 (Sec. 5.1, App. B)

과제	로봇	시연	학습/시험	성공 기준 (Table 4)	MPC 예산 (Table 5)
HOT3D	DG-5F 손만 (26-D)	사람 동작 780개 → 증강 360,150개, 약 2 s	28,480 / 1,000	150 mm, 40°, 침투 ≤ 5 mm, 쥐는 힘 ≥ 1 N, 손가락 힘 p95 ≤ 100 N	300 × 10 = 3,000
ShapeFilter	FR3 + DG-5F (29-D)	mocap 605개 → 자세 증강 3,037개, 11.0 s	2,448 / 589	십자 부품 중심이 삽입 영역(16 cm 상자) 안. 구멍 여유 1.96 mm	500 × 6 = 3,000
NIST	양팔 FR3 + DG-5F (29-D)	mocap 358개, 29 s	294 / 64	수평 오차 < 1 mm, 8 mm 이상 체결, 축 오차 < 0.003 rad	3단계 합계 3,440

- 기준선은 MPPI, DIAL-MPC, iCEM, CMA-ES이고, 모두 같은 비용 J를 씁니다(App. B.3). ShapeFilter에는 CEM을 추가했습니다(Table 10).
- 비용은 CPU core-s 또는 core-h입니다. GNR 추론과 시뮬레이션 재생을 포함합니다.

3.2 HOT3D: 다양성 축 (Sec. 5.2, Table 1, Fig. 3, App. D)

시험 집합: 학습에 없던 물체(203), 학습에 없던 파지(338), 학습에 없던 기하 변형(459)으로 1,000개를 뽑았습니다. DIAL-MPC 기준으로 272개가 MPC-success, 728개가 MPC-failure입니다.

방법	S	core-s	전체 SR	MPC-success SR	MPC-failure SR	MPC-failure E_pos (mm)
IK	1	39.0	18.3	50.0	6.5	54.5
DIAL-MPC	3,000	2,689	27.2	100	0	36.3
MPPI	3,000	1,850	27.7	71.3	11.4	36.9
iCEM	3,000	3,862	34.7	78.7	18.3	26.0
CMA-ES	3,000	2,913	32.0	72.8	16.8	32.9
GNR	1	48.9	18.1	48.2	6.9	54.5
GNR	64	90.9	49.6	89.0	34.9	37.1
GNR	256	354.1	56.2	92.6	42.6	36.2

해석:

- **GNR S=1은 IK와 같습니다**(18.1% 대 18.3%). 생성기 한 번의 출력이 IK보다 낫지 않고, 이득은 전부 표본을 늘리고 시뮬레이터로 거르는 데서 나옵니다. 이 구조는 `03` §2에서 다룹니다.
- **DIAL-MPC의 100%/0%는 정의입니다.** 시험 집합을 DIAL-MPC 결과로 나눴으므로 이 행은 다시 돌린 결과가 아니라 분할 레이블입니다(`04` C4). 공정한 비교 대상은 MPPI, iCEM, CMA-ES이고, 그중 가장 좋은 iCEM이 34.7%입니다.
- 추적 오차가 낮다고 성공하지는 않습니다. iCEM의 MPC-failure 쪽 E_pos 중앙값은 26 mm로 GNR보다 작지만 SR은 18%입니다. 저자들은 최적화가 추적 오차는 작지만 파지·접촉 조건을 어기는 국소해에 빠진다고 설명합니다.
- **OOD 부분집합(Fig. 3, App. D.3):** S=1 → 256에서 물체 12.8 → 44.3%, 파지 10.1 → 42.9%, 기하 26.4 → 71.2%로 오릅니다. MPC-failure 중 31.9-58.4%를 풉니다. 역산하면 부분집합별 MPC-success 비율이 물체 18%, 파지 20%, 기하 36%로 크게 다릅니다(`04` C3).

3.3 데이터셋 확장과 비용 (Sec. 5.2, App. D.4, G)

- DIAL-MPC는 360,150개 중 95,344개(26.47%)를 풀었고 390k core-h를 썼습니다(512코어로 31.7일).
- 28k로 학습한 GNR을 MPC-failure 264,806개에 S=256으로 적용하면 127,136개(48.01%)를 더 풉니다. 성공한 표본에서 추출을 멈춥니다(D.4). 합치면 222,806개(Table 8에서는 222,480개)이고, 물체 기하는 2,756 → 3,298개입니다.
- 같은 모델은 MPC-success 95k에서 89.30%, 그중 학습에 안 쓴 시연에서 88.91%를 냅니다.
- **실제 배치 시나리오(App. G.2):** MPC로 시드 28,480개를 얻으려면 약 107,740번 시도해야 하고 116.1k core-h가 듭니다. 나머지 331,670개에 GNR(S=256, 354.1 core-s)을 쓰면 32.6k core-h입니다. 합계 148.7k core-h로 215,062개를 얻으므로, 전부 MPC로 돌린 경우보다 2.62배 쌉니다.

3.4 ShapeFilter와 NIST: 정밀도·긴 시간 축 (Sec. 5.3, Fig. 4-5, Table 2, App. E-F)

ShapeFilter(십자 부품 삽입):

- IK는 3.7%입니다. Open-loop GNR은 S=1에서 52.4%, S=64에서 97.4%, S=256에서 99.3%입니다. S=64에서 MPC(3,000표본, 92.9%)를 넘고, 이때 CPU 비용은 MPC의 1.6%라고 합니다.
- GNR+MPC는 M=1-6 반복에서 98.4-99.3%이고, 같은 M에서 IK로 시작한 MPC보다 항상 높습니다(Fig. 4a).
- 시드 수 N을 줄여도 test-time scaling이 메웁니다. N=50으로 학습해도 S=1 11.5%에서 S=256 99.5%까지 오릅니다(Fig. 5a, Table 12).
- Table 2: MPC 2,500표본 93.7%, 10,124 core-s 대 GNR 77표본 94.7%, 678 core-s로 14.9배 차이입니다. 데이터셋 3,037개를 만드는 데 MPC는 8,541 core-h(93.7%), N=50 시드 + GNR S=77은 765.6 core-h(94.8%)로 11.2배 차이입니다.

NIST(M8 노브 체결, 29초):

- Open-loop GNR은 S=256에서도 4.7%입니다(IK 0%). 긴 시간 동안 오차가 쌓이기 때문에 폐루프 보정이 필요합니다.
- GNR+MPC(N=294)는 MPC 표본 2,296개로 92.2%에 도달합니다. IK에서 시작한 MPC는 3,440개가 필요하므로 33.3% 적습니다. N=100이면 90.6%이고 38.8% 적습니다.
- Table 2: MPC 3,440표본 92.2%, 795,777 core-s 대 GNR+MPC 1,976표본 89.1%, 266,199 core-s로 3.0배 차이입니다. 데이터셋 기준으로는 79,136 대 44,517 core-h로 1.8배 차이입니다.

3.5 부록 실험 (App. E.4, H, I)

- **조건 절제(Table 11):** ShapeFilter에서 자세(State), 점군(Geometry), 둘 다를 조건으로 주어도 S=1 50.4 / 50.9 / 53.5%, S=256 99.3 / 99.5 / 99.3%로 거의 같습니다.
- **잔차 대 절대(Fig. 12):** 학습 사분면 1·2·4개일 때 잔차 예측은 S=1에서 34.7 / 42.4 / 45.8%, S=128에서 92.2 / 96.1 / 94.9%입니다. 절대 예측 수치는 그림에만 있고, 차이는 S가 작을 때 크다고 씁니다.
- **다중 embodiment(Table 16):** SPIDER 벤치마크의 손 5종(Ability, Allegro, Inspire, Schunk, XHand)에서 147개로 GNR 하나를 학습했습니다(52-D 공통 행동). 시험 100개에서 S=128·256·384로 90·93·95%이고, SPIDER(최적화당 최대 16,384-32,768표본)는 96%입니다.
- **행동 복제(App. I.1):** 같은 크기(500 에피소드, 10.9 s)의 MPC, GNR+MPC, GNR 데이터로 203M 파라미터 MLP 정책을 학습했습니다. 체크포인트 10개 중 최고 SR이 37.5%, 43.75%, 43.75%입니다(시험 32회). 과제 이름은 적혀 있지 않지만 에피소드 길이로 보아 ShapeFilter입니다.
- **데이터 flywheel(App. I.2):** ShapeFilter에서 시드 200개로 시작해, GNR이 실패한 시연을 MPC로 풀어 학습 집합에 더하기를 4회 반복했습니다(200 → 749개). 시험 256개에서 best-of-8은 55.2 → 74.4%, best-of-16은 68.4 → 84.8%입니다.

---

4. Limitations

4.1 저자가 밝힌 한계

별도 한계 절은 없습니다. 결론에서 실물 전이(sim-to-real)를 향후 과제로 들고, NIST처럼 긴 과제에서는 open-loop 생성만으로 부족하다고 인정합니다(Sec. 5.3, 6). 모든 결과는 시뮬레이션이고 실물 로봇 실험은 없습니다.

4.2 분석하면서 추가로 발견한 한계와 의문점

1. **"MPC 27.2%"는 비교 결과가 아니라 시험 집합의 분할 비율입니다.** 1,000개를 DIAL-MPC 성공 272개와 실패 728개로 골랐으므로 DIAL-MPC 행은 정의상 100%/0%입니다. 초록의 "56.20% vs 27.20%"는 같은 조건의 재실행 비교가 아닙니다. 독립적으로 다시 돌린 기준선 중 최고는 iCEM 34.7%입니다. 프로젝트 페이지의 Table 1 각주는 "100% and 0% by construction"이라고 밝히지만, 논문의 표 각주와 초록에는 이 설명이 없습니다(`04` C1, C4).
2. **"시연" 수는 최적화 시도 단위입니다.** 기하 변형 쪽 173,840개는 (동작, 변형) 쌍 7,280개 × 최적화 시드 24개입니다. 쌍 단위 MPC 성공률은 37.4%인데 시도 단위로는 27.6%입니다. MPC-failure 12.6만 개 중 1.7만 개(13.8%)는 다른 시드로 MPC가 이미 푼 쌍입니다. GNR에게는 시드가 다른 시도 24개가 같은 조건이므로, "MPC가 실패한 127k개를 풀었다"는 서로 다른 문제 127k개가 아닙니다. 223k 궤적을 기하 3.3k개와 사람 동작 780개에서 만들었다는 점도 같이 봐야 합니다. Table 3에서 다른 연구의 시연 수와 나란히 놓은 비교는 단위가 다릅니다(`04` C5, `03` §4).
3. **이득은 거부 표집에서 나오고, 꼬리가 무겁습니다.** S=1은 IK 수준입니다. MPC-failure 쪽 pass@S를 시연별 성공 확률 분포로 맞추면 Beta(a ≈ 0.12-0.14)입니다. 표본을 두 배로 늘려도 실패가 약 8%만 줄어들고, S=1024로 외삽해도 49-56%입니다(`03` §2).
4. **비용 수치의 출처가 섞여 있습니다.** 7.6배는 Table 1 단가이고 Table 14 단가로는 9.9배입니다. 2.62배는 MPC에 Table 14 단가를, GNR에 Table 1 단가를 썼습니다. 반대로 GNR 확장 비용은 조기 종료를 무시한 상한이라, 조기 종료를 반영하면 26.0k가 아니라 약 16.8k core-h로 추정됩니다(`04` C6, `03` §2.3).
5. **ShapeFilter 수치의 분모와 판이 섞여 있습니다.** Table 10의 일부 기준선은 64개에서만 정수 비율이 되고, S=1 값이 본문 52.4%, Table 11 50.42%로 다릅니다. Table 2의 GNR 행(77표본)은 2,448개가 아니라 N=50 모델입니다. "CPU 비용 1.6%"는 조기 종료를 할 때만 성립합니다. 표본 64개를 다 쓰면 약 21%입니다(`04` C7).
6. **"비슷한 SR"의 근거가 약합니다.** NIST Table 2는 59/64 대 57/64(p = 0.76)이고, 표본 수도 본문 2,296개가 아니라 1,976개입니다. 다중 embodiment는 GNR 95% 대 SPIDER 96%로 GNR이 이기지 못합니다. BC는 12/32 대 14/32(p = 0.80)이고 체크포인트 10개 중 최고 값입니다(`04` C8-C10).
7. **HOT3D 성공 기준이 느슨합니다.** 위치 150 mm는 CHORD 관례입니다. MPC-failure 쪽 E_pos 중앙값이 26-55 mm라서 위치 조건은 거의 걸러 내지 못하고, 실제로는 침투·쥐는 힘·손가락 힘 조건이 성공을 가릅니다. ShapeFilter의 "1.96 mm"는 구멍 여유이고, 판정은 부품 중심이 16 cm 삽입 영역 안에 있는지입니다. Table 3의 "Precision (mm)" 열은 성격이 다른 기준을 한 줄에 놓았습니다(`04` C12).
8. **접촉력 라벨은 시뮬레이터 설정의 산물입니다.** 접촉 강성을 기본값의 1/10로 낮추고 median filter를 적용했습니다. 실물 힘과 대조한 결과는 없습니다(App. C.4).

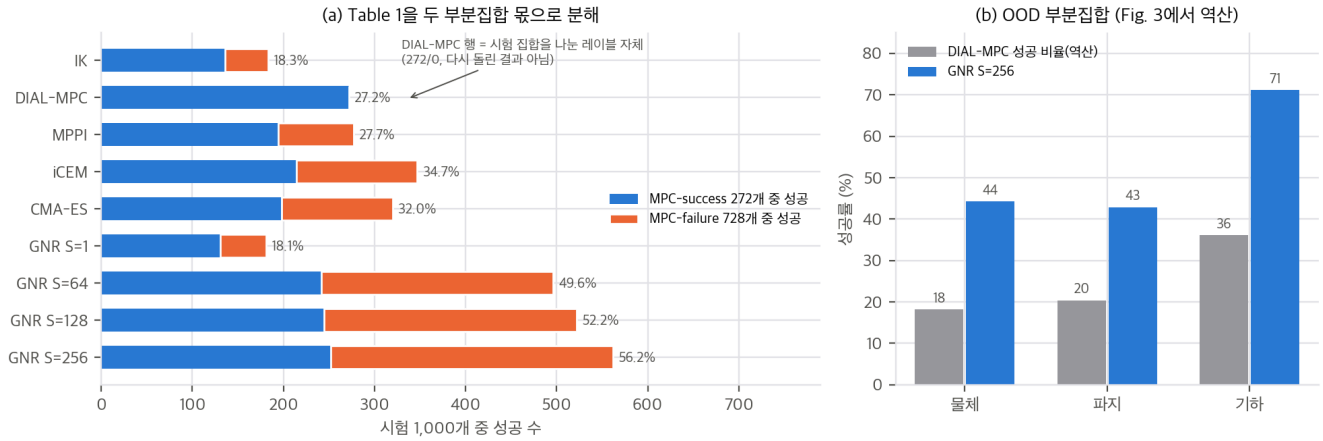
9. **생성기가 시드 생성기 밖으로 얼마나 나가는지는 열린 문제입니다.** 학습 데이터는 모두 DIAL-MPC 성공 궤적이므로 GNR의 제안 분포는 DIAL-MPC 해의 분포를 닮습니다. MPC-failure를 푸는 이유가 "MPC 예산·시드 운 부족"인지 "MPC가 못 가는 해"인지는 구분되지 않았습니다. toy에서는 학습 범위 안 보간은 잘 되고 밖으로 외삽하면 실패했습니다(`03` §3).
10. **더 단순한 기준선이 없습니다.** 생성 모델이 꼭 필요한지 보려면 결정론적 회귀(잔차 MLP)나 최근접 시드 잔차 검색과 비교해야 하는데, 논문에는 둘 다 없습니다. toy에서는 최근접 시드 + 국소 보정만으로도 학습 범위 안에서 MPC를 넘었습니다(`03` §3).
11. **코드, 가중치, 데이터, mocap 시연이 모두 공개되지 않았습니다.** 시뮬레이터(SuperDex)와 DG-5F·FR3 모델은 공개되어 있습니다(`02` §5).

03. 수치 재계산과 해석

GNR의 주장은 대부분 "표본 S개 중 하나라도 통과하면 성공"이라는 pass@S와 CPU 비용 회계에 기대고 있습니다. 이 문서는 (1) Table 1의 구조, (2) pass@S 곡선이 왜 그렇게 생겼는지, (3) "가능한 해가 저차원 manifold 근처에 있다"는 가설의 toy, (4) "시연" 단위, (5) 비용 회계를 다룹니다. 모든 계산은 CPU 단일 스레드 numpy·scipy이고 각 스크립트는 수 초에서 수십 초 걸립니다.

스크립트	결과 파일	내용
`analysis/table_check.py`	`table_check_results.json`	표·본문·부록 재계산 C1-C12 (`04` §1)
`analysis/scaling_toy.py`	`scaling_toy_results.json`	pass@S 이질성 모델 적합, 조기 종료 비용
`analysis/manifold_toy.py`	`manifold_toy_results.json`	manifold 가설 toy: 생성기 대용 대 annealed CEM
`analysis/plot_figures.py`	`report/figures/*.png`	이 문서의 그래프 네 장

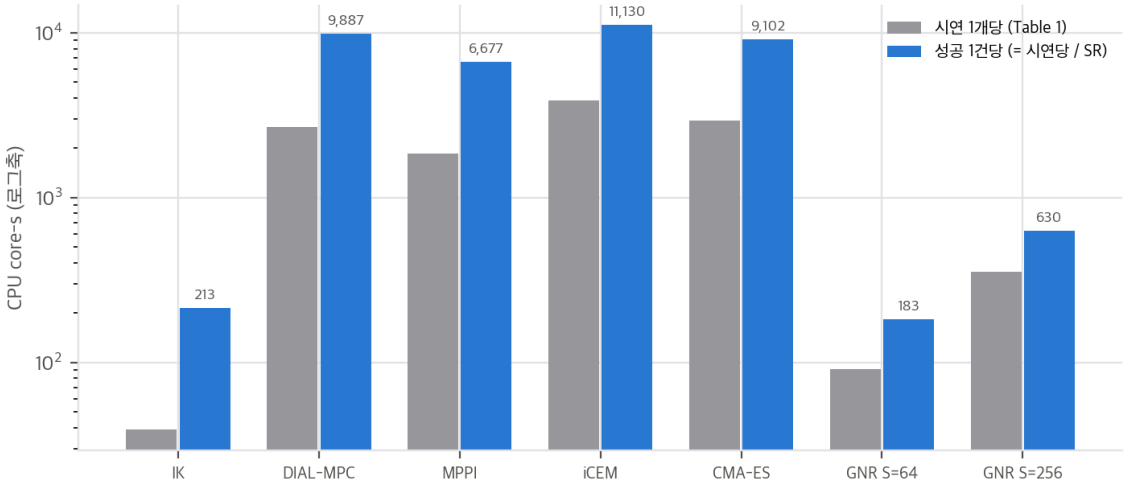
1. Table 1을 두 부분집합으로 다시 읽기 (`04` C2-C4)



출처: Table 1, Fig. 3, 부록 D.3. 부분집합별 MPC 성공 수 37·69·166의 합이 정확히 272. analysis/table_check.py

그림 1. (a) Table 1의 각 행을 MPC-success 272개와 MPC-failure 728개 안의 성공 수로 나눈 것. 9개 행 모두 정수 개수로 정확히 맞습니다. (b) Fig. 3과 부록 D.3에서 역산한 OOD 부분집합별 DIAL-MPC 성공 비율과 GNR S=256.

- 전체 SR = 0.272 × SR_success + 0.728 × SR_failure가 9개 행 모두 반올림 안에서 맞고, 성공 수가 정수입니다(예: GNR S=256 = 252 + 310 = 562개).
- **DIAL-MPC 행은 분할 레이블입니다.** 272/1,000 = 27.2%이고, MPC-success에서 100%, MPC-failure에서 0%는 정의입니다. 모집단 MPC 성공률은 26.47%(Table 8)라서 시험 집합은 무작위 추출과 비슷합니다. 다만 초록의 "56.20% vs 27.20%"는 "같은 조건에서 MPC를 다시 돌린 결과"와는 다릅니다. 같은 MPC를 다른 시드로 다시 돌리면 MPC-failure의 일부를 풉니다(§4). 프로젝트 페이지의 표 각주는 이 행이 정의상 100%/0%라고 밝히지만, 논문 Table 1에는 그 각주가 없습니다.
- 독립적으로 다시 돌린 기준선(MPPI, iCEM, CMA-ES)은 MPC-success에서 71-79%만 다시 풉니다. 같은 계열 최적화기라도 시드와 잡음에 따라 결과가 크게 갈립니다. 이 사실은 MPC-failure가 "풀 수 없는 문제"가 아니라 상당수가 "이번 실행에서 못 푼 문제"라는 뜻입니다.
- OOD 부분집합 세 개의 MPC 성공 수를 역산하면 37·69·166개이고 합이 정확히 272입니다. 기하 변형 부분집합은 MPC도 36%를 풀 만큼 쉽고, 물체·파지 OOD는 18-20%로 어렵습니다. GNR의 OOD 성능(44·43·71%)도 같은 순서입니다. 즉 부분집합 간 차이는 GNR의 일반화보다 문제 난이도를 더 많이 반영합니다.



HOT3D 시험 1,000개. 같은 방법의 Table 14 단가는 Table 1보다 1.11-1.52배 큼 → DIAL/GNR 비는 Table 1 기준 7.6배, Table 14 기준 9.9배.

그림 2. HOT3D 시험 1,000개에서 시연 1개당 비용과 성공 1건당 비용(= 시연당 비용 / SR). GNR S=64는 성공 1건당 183 core-s로, DIAL-MPC 9,887 core-s의 1/54입니다.

성공 1건당 비용으로 보면 차이가 더 큽니다. 비율로 계산되는 이 지표는 시험 집합 분할의 영향을 받지 않습니다. 다만 IK도 213 core-s로 매우 싸다는 점을 같이 봐야 합니다. GNR S=64(183 core-s)가 IK보다 싼 이유는 IK의 성공률이 낮아서입니다.

2. Test-time scaling의 구조

2.1 유도

시연 i를 GNR이 표본 하나로 풀 확률을 p_i라고 합시다. 표본들이 조건부 독립이면 pass@S = 1 − E[(1 − p)^S]입니다. 모든 시연의 p가 같다면 S=1 성공률 p_1에서 곡선이 정해집니다. HOT3D MPC-failure는 p_1 = 6.87%이므로 pass@64가 98.9%여야 하는데, 실제로는 34.9%입니다. **시연마다 p가 크게 다릅니다.**

p의 분포를 "아예 못 푸는 비율 π_0 + 나머지는 Beta(a, b)"로 두면 닫힌 꼴이 나옵니다.

$E[(1-p)^S \mid \text{Beta}] = B(a, b+S) / B(a, b)$

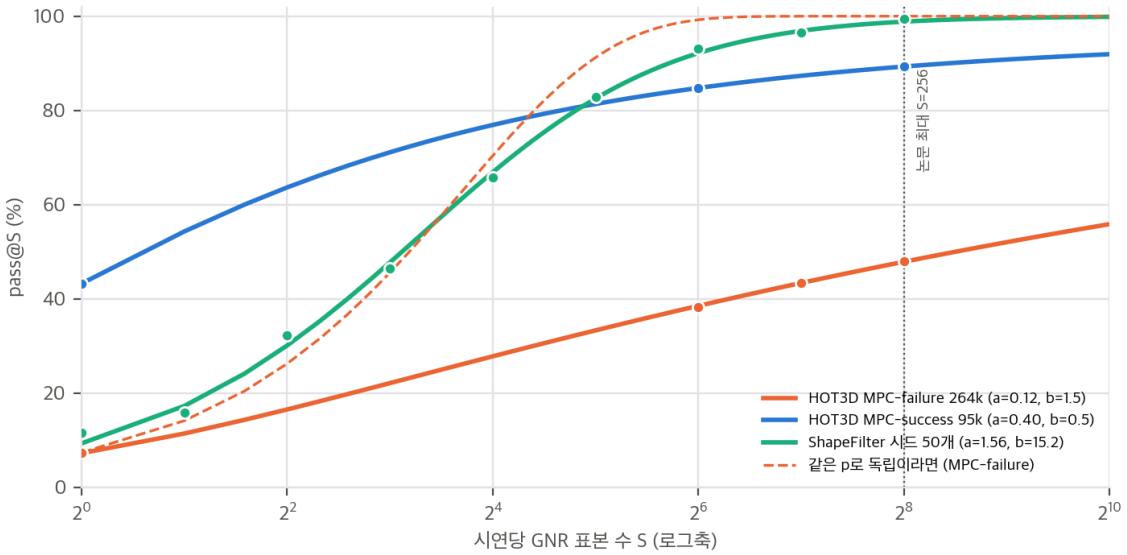
$\text{pass@}S = (1-\pi_0) \cdot (1 - B(a, b+S)/B(a, b))$

S가 크면 $B(a, b+S)/B(a, b) \approx \Gamma(a+b)/\Gamma(b) \cdot S^{-a}$이므로 실패율이 **$S^{-a}$의 거듭제곱 꼬리**로 줄어듭니다. 표본을 두 배로 늘릴 때마다 (π_0를 뺀) 실패가 2^{-a}배가 됩니다. a는 "p가 0 근처에 얼마나 몰려 있는가"를 나타내는 하나의 숫자입니다.

조기 종료(성공하면 추출 중단)를 할 때 기대 표본 수는 다음과 같습니다.

$E[\min(T, S)] = \Sigma_{k=0}^{S-1} P(\text{처음 } k\text{개 모두 실패}) = \Sigma_{k=0}^{S-1} [\pi_0 + (1-\pi_0) \cdot B(a, b+k)/B(a, b)]$

2.2 적합 결과 (`scaling_toy.py`)



점: 논문 값(부록 D.4, Table 12에서 역산). 선: 성공 확률 p ~ Beta(a,b)(+0 질량) 적합. a<1이면 p가 0 근처에 몰려 표본을 늘려도 느리게 오름. analysis/scaling_toy.py

그림 3. 논문 값(점)과 이질성 모델 적합(선). 주황 점선은 모든 시연의 p가 같다고 가정한 예측입니다. HOT3D MPC-failure는 a = 0.12라서 로그축에서도 거의 직선으로 천천히 오릅니다.

곡선 (출처)	π0	a	b	RMSE (%p)	시연당 평균 p	S=1024 외삽	조기 종료 시 기대 표본 (S=256)
HOT3D 시험 MPC-failure 728 (Table 1)	0.21	0.14	1.46	0.35	0.069	48.8%	163
HOT3D 360k MPC-failure (D.4)	0.00	0.12	1.55	0.18	0.072	55.8%	151
HOT3D 시험 MPC-success 272 (Table 1)	0.04	0.46	0.46	0.42	0.48	94.2%	29
HOT3D 360k MPC-success (D.4, 점 3개)	0.05	0.40	0.48	0	0.43	91.9%	38
ShapeFilter State (Table 11)	0.00	0.84	0.81	0.31	0.51	99.7%	8
ShapeFilter 시드 N=2,448 (Table 12 역산)	0.00	0.78	0.70	0.21	0.53	99.7%	8
ShapeFilter 시드 N=50 (Table 12 역산)	0.00	1.56	15.2	1.33	0.09	99.9%	23

읽는 법:

1. **HOT3D MPC-failure의 꼬리 지수 a ≈ 0.12-0.14입니다.** 표본을 두 배로 늘려도 실패가 8-9%만 줄어듭니다. S=256 → 1024(4배)로 외삽해도 49-56%입니다. 점이 4개뿐이라 π0(점근선)는 정해지지 않습니다. 시험 집합 적합은 π0 = 0.21, 360k 적합은 0이 나옵니다. 믿을 수 있는 것은 "느린 거듭제곱 꼬리"라는 모양이고, 점근선 값은 아닙니다.
2. **ShapeFilter는 이질성이 작습니다**(a ≈ 0.8). 두 배마다 실패가 약 42% 줄고, 조기 종료하면 평균 8개면 됩니다.
3. **시드 N=50은 모양이 다릅니다.** 평균 p는 0.09로 낮지만 a = 1.56, b = 15로 p가 모든 시연에 고르게 낮습니다. 그래서 S를 늘리면 빨리 99%에 닿습니다. 데이터가 적은 생성기는 "모든 시연에 넓고 흐린 제안"을 하고, 데이터가 많은 생성기는 "대부분에 날카롭고 일부에 거의 0인 제안"을 합니다. 논문의 "test-time scaling이 data scaling을 보완한다"(Sec. 5.3)는 ShapeFilter처럼 p가 0으로 몰리지 않는 과제에서만 성립합니다.
4. MPC-failure의 무거운 꼬리는 두 가지로 해석할 수 있습니다. (i) 복원 오차나 embodiment 차이로 레퍼런스 자체가 실행 불가능한 시연이 섞여 있다. (ii) 생성기 분포가 해에 거의 닿지 않는다. 논문 데이터만으로는 구분할 수 없습니다. 실패 시연에 MPC 예산을 크게 늘려 보는 실험이 있으면 구분됩니다.

제안: pass@256 하나 대신 (a, 평균 p)를 함께 보고하면 "표본을 더 쓰면 얼마나 오르는가"를 예측할 수 있습니다.

2.3 조기 종료와 비용

Table 1의 비용은 S에 거의 비례합니다(S=64 → 128 → 256에서 표본당 1.37 core-s). 그래서 조기 종료 없이 잰 값으로 봅니다. 부록 D.4는 확장 실험에서 "성공하면 추출을 멈춘다"고 쓰지만, 비용 추정(App. G.2)은 "모든 시연에 전체 예산을 쓴다"고 명시하고 354.1 core-s를 그대로

곱했습니다. 즉 **논문의 GNR 비용은 상한**입니다.

표본당 1.197 core-s, 고정 비용 47.7 core-s로 근사하고(S=1 → 256 기울기) 적합한 분포의 기대 표본을 쓰면 다음과 같습니다.

항목	논문 (전체 예산)	조기 종료 추정
MPC-failure 264,806개 확장	26,047 core-h	약 16,800 core-h
실제 배치 시나리오의 GNR 부분 (331,670개)	32,623 core-h	약 18,600 core-h
실제 배치 시나리오 전체 대비 MPC 절감	2.62배	약 2.90배

반대로 이 시나리오에서 MPC 시드 비용(116.1k core-h)이 전체의 78%입니다. GNR 비용이 0이어도 절감은 390.0 / 116.1 = 3.36배를 넘지 못합니다. 더 줄이려면 시드 수를 줄여야 하고, 그 대가는 §2.2의 a가 작아지는 것(무거운 꼬리)으로 나타날 가능성이 큽니다.

3. manifold 가설 toy (`manifold_toy.py`)

질문: 가능한 해가 조건 c에 매끄럽게 의존하는 저차원 집합 근처에 있으면, 풀린 시연에서 배운 조건부 생성기가 시연마다 처음부터 찾는 sampling 최적화보다 적은 표본으로 푸는가? 그 이득은 어디까지 유지되는가?

설정: 물리 시뮬레이션이 아니라 기하만 남긴 장난감입니다.

- 조건 $c \in [-1, 1]^2$(물체 크기, 접근 각 같은 시연 요약). IK 기준 $\bar{u}(c) \in R^{24}$는 c에 따라 크게 변합니다.
- 가능한 잔차는 3차원 잠재 구조 $\Delta^*(c) = A \cdot h(c)$에 **두 가지 파지 모드**(오프셋 B)를 더한 것입니다. 어느 모드든 $\| \Delta - \Delta^*_m(c) \| < 0.45$이면 성공입니다. 표본 하나를 평가하는 것이 시뮬레이션 롤아웃 한 번입니다.
- MPC: $\Delta = 0$(IK)에서 시작하는 annealed CEM, 300표본 × 10반복 = 3,000표본, 첫 성공에서 멈춤.
- 시드: 조건 1,600개를 MPC로 풀어 성공한 것만 씁니다(66.9%). 보간 OOD 시험을 위해 원판(중심 (0.3, −0.2), 반지름 0.35) 안의 시드는 뺐고, 930개가 남았습니다.
- 생성기 대응: k-최근접 시드(k=12)를 모드별로 나누고, 모드마다 국소 선형 회귀로 대상 조건의 해를 예측한 뒤 잡음($\sigma = 0.06$)을 더해 S개를 뽑습니다. **flow matching이 아니라 가장 단순한 비모수 조건부 생성기**입니다. 절대 제어 버전은 같은 일을 $u = \bar{u} + \Delta$ 공간에서 합니다.

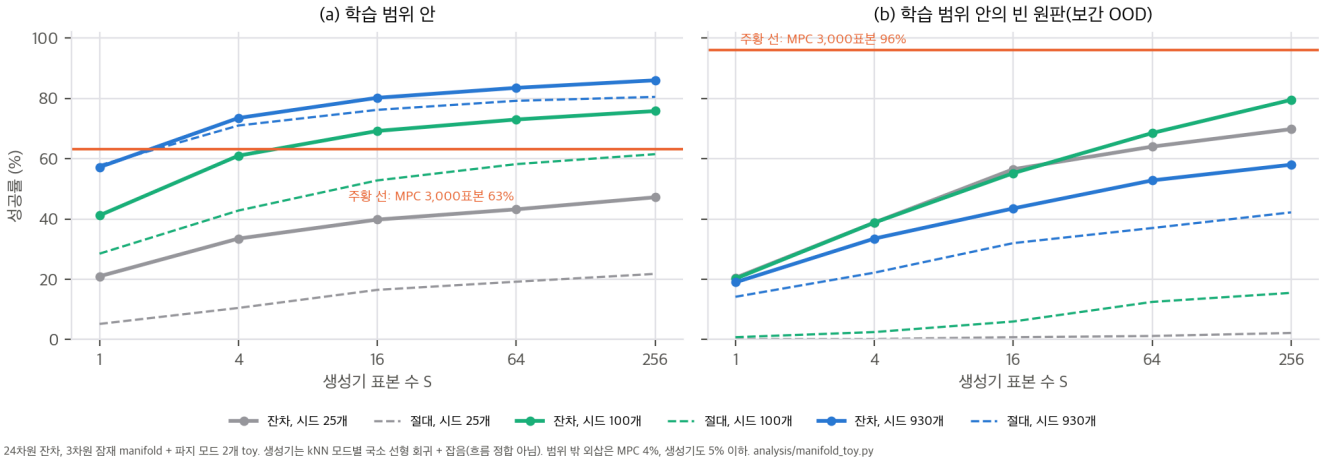


그림 4. 생성기 대응의 pass@S. (a) 학습 범위 안, (b) 학습 범위 안의 빈 원판(보간 OOD). 주황 선은 MPC 3,000표본. 학습 범위 밖 외삽($c1 \in [1, 1.6]$)은 MPC 4.5%, 생성기 4.5% 이하라 그리지 않았습니다.

시드 수 N	범위 안 pass@1 / @16 / @256 (잔차)	범위 안 @256 (절대)	빈 원판 @256 잔차 / 절대	범위 밖 @256	MPC-failure 147개 중 생성기가 푼 비율 (@256)
25	21.0 / 39.8 / 47.2	21.8	69.8 / 2.2	0	2.7%
100	41.2 / 69.2 / 75.8	61.5	79.5 / 15.5	0	38.1%
400	54.8 / 71.8 / 80.0	73.2	74.0 / 45.5	3.2	46.3%
930	57.2 / 80.2 / 86.0	80.5	58.0 / 42.2	4.5	61.9%

시드 수 N	범위 안 pass@1 / @16 / @256 (잔차)	범위 안 @256 (절대)	빈 원판 @256 잔차 / 절대	범위 밖 @256	MPC-failure 147개 중 생성기가 푼 비율 (@256)
MPC 3,000표본	63.2		96.0	4.5	(정의상 0%)

해석:

1. **범위 안에서는 상각이 작동합니다.** N=930이면 표본 4개(73.5%)로 MPC 3,000표본(63.2%)을 넘습니다. 같은 CEM을 192표본으로 줄이면 0%입니다. 논문의 "MPC 표본의 8.5%로 MPC보다 높다"와 같은 방향입니다.
2. **"MPC가 못 푼 문제를 푼다"는 MPC 밖의 해를 찾았다는 뜻이 아닐 수 있습니다.** toy의 모든 조건에는 해가 있으므로 MPC 실패는 전부 예산·국소 탐색 실패입니다. 생성기는 이웃 시드에서 해를 옮겨 와서 이 중 62%를 풉니다. 논문의 MPC-failure 42.6%도 같은 메커니즘(주변 시연의 해를 옮겨 옴)으로 설명할 수 있습니다.
3. **잔차 예측이 절대 예측보다 낫고, 데이터가 적거나 보간 OOD일 때 차이가 큽니다**(N=100 빈 원판에서 79.5% 대 15.5%). $\bar{u}(c)$가 c에 따른 큰 변화를 이미 담고 있어서, 잔차가 남기는 부분이 더 작고 매끄럽기 때문입니다. App. E.4와 같은 방향입니다.
4. **학습 범위 밖 외삽은 실패합니다**(4.5% 이하). 생성기는 시드의 지지 집합 밖으로 나가지 못합니다. 논문의 OOD(새 물체·파지·기하)는 이 toy의 "빈 원판"에 가까운 보간일 가능성이 큽니다. 물체·파지 OOD에서도 MPC 성공 비율이 18-20%로 낮지 않습니다(§1).
5. **두 모드의 평균을 내는 결정론적 회귀는 0%입니다**(모드 간격의 절반이 ε의 4.4배). 해 집합이 여러 모드일 때 생성 모델이 필요한 이유입니다. 그런데 **논문에는 결정론적 회귀나 최근접 시드 검색 같은 더 단순한 기준선이 없습니다.** 이 toy의 생성기 대용이 바로 "최근접 시드 잔차 + 국소 보정" 검색 기준선인데, 범위 안에서는 MPC를 넘었습니다. flow matching이 검색보다 얼마나 나은지는 논문만으로 알 수 없습니다.
6. **한계:** 빈 원판에서는 N=400(74.0%)보다 N=930(58.0%)이 낮습니다. 데이터가 촘촘해지면 k=12 이웃이 원판 가장자리 한쪽에 몰려 국소 선형 외삽이 나빠지는 kNN 특유의 현상입니다. 따라서 N에 대한 정량 결론은 이 toy로 내리지 않습니다. MPC가 빈 원판에서 96%를 내는 것도 이 영역이 쉬워서입니다.

4. "시연" 단위와 데이터셋 크기 (`04` C5)

부록 C.5와 Table 8을 맞춰 보면, 기하 변형 쪽 "시연"은 (사람 동작, 기하 변형) 쌍에 최적화 시드를 곱한 **최적화 시도**입니다.

단위	개수	MPC 성공
동작-변형 쌍	728 × 10 = 7,280	2,723쌍 (37.4%)
시도 = 쌍 × 시드 24	174,720 (평가 173,840, 880개 누락)	47,918 (27.6%)
MPC가 푼 쌍 안의 시도	2,723 × 24 = 65,352	47,918 (73.3%)

- 기하 변형 쪽 MPC-failure 125,922개 중 **17,434개(13.8%)는 다른 시드로 MPC가 이미 푼 쌍**입니다.
- GNR의 조건 c = ($\bar{x}$, o)는 최적화 시드와 상관이 없습니다. 같은 쌍의 시도 24개는 GNR에게 같은 문제를 24번(S=256씩) 푸는 것입니다. 기하 변형 쪽에서 GNR이 더 푼 52,745개는 서로 다른 문제 52,745개가 아닙니다.
- 원래 기하 쪽 186,310개도 사람 동작 780개에서 나왔습니다(동작당 평균 239개: 자세·시점·질량·마찰 증강과 시드).
- 궤적 수는 95,344 → 222,480개(+133%)로 늘지만, 서로 다른 물체 기하는 2,755 → 3,298개(+19.7%)이고 사람 동작은 780개 그대로입니다.

정리: "223k 시연, 3.3k 기하"는 사실이지만, 다양성의 실제 단위는 사람 동작 780개 × 기하 3.3k개입니다. Table 3에서 DexTrack 3,585개, SPIDER 2,885개 같은 수와 나란히 놓으면 단위가 다릅니다. 본문(222,806개, 기하 2,756개, MPC 성공률 26.43%)과 Table 8(222,480개, 2,755개, 26.47%)도 조금씩 다릅니다.

5. 비용 회계 (`04` C6, C7)

- **단가 출처:** 같은 방법의 Table 14 단가(core-h/에피소드)가 Table 1(core-s)보다 1.11-1.52배 큽니다. DIAL-MPC/GNR 비는 Table 1로 7.6배, Table 14로 9.9배입니다. 2.62배(App. G.2)는 MPC에 Table 14, GNR에 Table 1을 썼습니다. Table 14로 통일하면 2.56배입니다. 어느 쪽이든 결론은 같고, 차이는 작습니다.

- **ShapeFilter "1.6%":** Table 12의 비용은 조기 종료 비용입니다. S가 커지면 증분이 포화합니다. N=2,448에서 S=1 → 2 증분(18.3 core-s)을 S=1 실패율(47.4%)로 나누면 GNR 표본 하나는 약 38.6 core-s입니다. MPC 롤아웃 하나(3.84 core-s, 20스텝 구간)의 약 10배이고, GNR 표본이 전체 에피소드를 재생하기 때문으로 보입니다. 따라서 S=64의 비용은 다음과 같습니다.
  - 조기 종료: 38.6 + 171.1 ≈ 210 core-s → MPC 3,000표본(11,520 core-s)의 **약 1.8%**. 논문 1.6%와 비슷합니다.
  - 조기 종료 없이 64개를 모두 쓰면: 64 × 38.6 ≈ 2,470 core-s → **약 21%**.
  - "1.6%"는 대부분의 시연이 처음 몇 개 표본에서 풀리기 때문에 나오는 수치입니다. ShapeFilter의 a ≈ 0.8(§2.2)이 이를 뒷받침합니다.
- **학습 비용:** ShapeFilter·NIST의 GPU 학습은 H100으로 0.01-0.5 GPU-h라 무시할 만합니다(App. G.1). HOT3D 모델(19.57M, A100)의 학습 시간은 보고되지 않았습니다.

추가 검증 (2026-11-11)

스크립트: `analysis/extra_tail_and_generators.py` → `analysis/extra_tail_and_generators_results.json` (numpy·scipy, 단일 스레드, 약 5초). 내부에서 `manifold_toy.py`의 함수를 재사용한다.

추가 검증 1: 실행 불가능한 시연의 섞임이 꼬리 지수를 만드는가

목적. §2.2의 HOT3D MPC-failure 꼬리 지수 a ≈ 0.14가 해결 불가능한 시연이 섞여 있어서 생기는지, 아니면 해결 가능한 시연의 p 분포 모양이 필요한지 본다.

설정(가정).

- 시연 집단의 p(시연 하나를 GNR 표본 하나로 풀 확률)를 두 부분으로 나눈다. 비율 f는 실행 불가능(p = 0)이고, 나머지 1 − f는 p ~ Beta(α, β)다.
- 그러면 pass@S = (1 − f)(1 − B(α, β + S)/B(α, β))이다. 표본 독립 가정은 §2.1과 같다.
- α ∈ {0.14, 0.5, 2}, f ∈ {0, 0.2, 0.4, 0.5}마다 S = 1의 성공률 6.87%에 맞도록 β를 정하고, S = 64·128·256의 관측값 34.89, 38.05, 42.58%(Table 1)와 비교했다.
- 비교로 §2.2의 3-모수 적합을 두 가지로 다시 했다. π0 자유, π0 = 0 고정.

결과.

f (실행 불가능)	α	β	예측 S = 64 / 128 / 256 (%)	RMSE (%p)
0	0.14	1.90	41.1 / 46.5 / 51.4	7.9
0.2	0.14	1.49	34.8 / 39.0 / 42.7	**0.5**
0.4	0.14	1.08	28.1 / 31.1 / 33.7	7.6
0.2	0.5	5.32	58.3 / 64.4 / 68.8	25.4
0.2	2.0	21.3	74.8 / 78.3 / 79.5	39.1
0.5	0.5	3.14	39.6 / 42.6 / 44.7	4.0
0.5	2.0	12.6	48.6 / 49.6 / 49.9	11.2

관측 곡선의 적합: π0 자유 → π0 = 0.21, a = 0.14, b = 1.46(RMSE 0.35%p). π0 = 0 고정 → a = 0.098, b = 1.30(RMSE 0.40%p).

결론.

- 실행 불가능한 비율만 섞고 해결 가능한 쪽을 날카롭게(α = 2) 두면 곡선이 나오지 않는다(RMSE 25-39%p). 곡선 모양에는 해결 가능한 시연 쪽 p가 0 근처에 많이 몰린 분포(α ≈ 0.14)가 필요하다.
- 두 요소가 함께 있을 때 가장 잘 맞는다. f = 0.2와 α = 0.14의 조합이 RMSE 0.5%p다. f = 0이면 7.9%p로 떨어진다.
- 따라서 §2.2에서 든 두 해석 중 (i) "실행 불가능한 시연 섞임"만으로는 부족하다. 실행 불가능한 시연은 필요하지만 충분하지 않다. 이 family 안에서는 꼬리 지수 a ≈ 0.1이 해결 가능한 시연의 p 분포가 가진 성질에서 온다고 보는 편이 맞다.

- π0를 0으로 고정하면 a가 0.098로 내려간다. 작은 a는 실패 질량을 꼬리에 흡수한 결과이기도 하다. 그래서 a만으로 "생성기 분포가 해에 닿지 않는다"는 결론을 내릴 수 없다.
- **한계.** Beta 혼합 가정, Table 1의 점 4개만 사용. 같은 곡선을 설명하는 다른 분포가 있을 수 있다. 시연별 p를 직접 재지 않았으므로 이 분해는 모형 안의 설명이다.

추가 검증 2: 조건부 생성기 비교(검색, 결정론적 회귀, 가우시안 혼합)

목적. §3의 manifold toy에서 최근접 검색 기준선이 다른 조건부 생성기보다 얼마나 나은지 본다. 비교 대상은 결정론적 회귀와, (c, Δ) 결합 공간에서 EM으로 적합한 가우시안 혼합 조건부 밀도다. 실행 불가능한 시드가 섞였을 때의 변화도 함께 본다.

설정(가정).

- manifold toy와 같은 장난감: D = 24, 잠재 차원 3, 모드 2, ε = 0.45. 시드는 MPC(3,000표본)로 푼 930개(보간 원판 밖).
- 오염 q = 0.2는 시드 187개의 Δ를 무작위 값(범위 ±2.5)으로 바꾼다. 실행 불가능한 시연에 해당한다.
- 검색: manifold toy의 최근접 잔차 방식(k = 12).
- 결정론적 회귀: 대상 조건에서 두 모드의 평균 한 점을 내고 잡음(σ = 0.06)을 더한다.
- 가우시안 혼합: K = 3, EM 300회(정규화 1e-3), 조건부 샘플링. 표본은 대상 조건마다 256개.
- 시험: 학습 범위 안 400개, 보간 원판 400개, S = 256. 시드를 바꾼 5회 반복(오염 위치와 EM 초기화가 달라짐)의 평균과 표준편차.

결과.

시험 집합	오염 q	생성기	pass@1	pass@16	pass@256
학습 범위 안	0	검색	60.0 ± 2.0	78.7	83.2 ± 0.3
학습 범위 안	0	가우시안 혼합	8.8 ± 1.1	55.0	79.0 ± 2.0
학습 범위 안	0	결정론적 회귀	0	0	0
보간 원판	0	검색	19.0 ± 0.9	50.4	67.7 ± 1.1
보간 원판	0	가우시안 혼합	7.3 ± 2.8	54.6	94.4 ± 7.3
학습 범위 안	0.2	검색	40.4 ± 0.8	62.8	67.3 ± 1.4
학습 범위 안	0.2	가우시안 혼합	2.7 ± 2.3	25.0	66.6 ± 8.0
보간 원판	0.2	검색	11.6 ± 1.7	33.7	46.2 ± 7.1
보간 원판	0.2	가우시안 혼합	1.9 ± 0.9	27.0	85.5 ± 6.8

결론.

- 결정론적 회귀는 모든 조건에서 0%다. 두 모드의 평균은 어느 모드에도 맞지 않는다. 생성 모델이 필요하다는 §3의 결론을 다시 확인한다.
- 가우시안 혼합은 S = 1에서 검색보다 훨씬 약하다(8.8% 대 60%). 표본을 늘리면 보간 원판에서 검색을 앞선다(256에서 94% 대 68%). 제안 분포가 넓고 흐린 생성기이므로, 표본당 비용이 큰 과제에서는 검색이 더 낫다.
- 시드 20%를 실행 불가능하게 바꾸면 두 방법 모두 떨어지지만, 보간 원판에서는 가우시안 혼합이 더 잘 버틴다(85.5 ± 6.8 대 46.2 ± 7.1). 학습 범위 안에서는 차이가 없다(66.6 ± 8.0 대 67.3 ± 1.4).
- 보간 원판에서 가우시안 혼합이 학습 범위 안보다 높게 나온 것은 toy의 특성으로 보인다(원판은 시드 사이 내부라 매끄러운 보간이 쉽다). 원인은 분리하지 않았다.
- **한계.** 시드 한 풀, 시드 변동 5회만 반영, K = 3과 k = 12 고정. flow matching은 쓰지 않았으므로 논문 모델과 비교하지 않았다. 검색과 가우시안 혼합의 차이를 논문 모델에 옮기는 근거는 되지 않는다.

추가 검증 3: 평면 3관절 팔 toy에서 잔차 flow matching은 절대 생성이나 검색보다 나은가

목적. §2의 pass@S 구조를 생성 모델 수준에서 확인합니다. 같은 과제에서 (i) 기준 행동(IK 한 번) 대비 잔차를 flow matching으로 학습한 생성기, (ii) 절대 행동을 flow matching으로 학습한 생성기, (iii) 가까운 학습 예시의 잔차를 옮기는 검색을 비교합니다.

설정(가정).

- 과제: 3관절 평면 팔(링크 0.5, 0.35, 0.25 m)의 관절각 $a \in R^3$가 끝점 $f(a) \in R^2$를 목표 c에 2 cm 안으로 맞추면 성공합니다. 시뮬레이터 판정은 정확한 f로 합니다. 해는 곡선(관절 3개, 제약 2개)이라 여러 해가 있습니다.
- 기준 행동 $a_0(c)$: 명목 자세 (0.3, 0.3, 0.3)에서 Jacobian 최소 노름 역변환 한 번.
- 학습 쌍 40,000개(관절각 균일 샘플 −2-2 rad), MLP 256폭 3층 flow matching, 1500 step, Euler 16 step. U-Net 대신 MLP를 썼습니다(3차원 행동에 시간 해상도 구조가 없다고 보았습니다). 시험 목표 500개, $S \in \{1 \cdots 256\}$, 시드 2개.
- 장치: CPU. 패키지: torch 2.15.0, numpy 1.26.4, Python 3.12.7. 실행 22초.

결과 (pass@S, 시드 2개 평균).

생성기	S=1	S=4	S=16	S=64	S=256	실패율 적합 a
잔차 flow	0.039	0.12	0.39	0.74	0.96	0.52
절대 flow	0.12	0.36	0.83	0.98	1.00	1.29
검색(잔차 옮기기)	0.94	0.97	0.98	0.98	0.98	0.20 (포화)

결론.

- 이 toy에서는 flow matching이 검색보다 낫지 않습니다. S = 1에서 검색 0.94 대 잔차 flow 0.04로 차이가 큽니다. 40,000개 학습 쌍이 시험 목표 근처를 촘촘히 덮어서 가까운 예시의 잔차가 거의 그대로 맞기 때문입니다. 검색 곡선은 S = 4 이후 0.98 근처에서 포화하므로 a 값을 해석하지 않습니다.
- 잔차 parameterization이 절대보다 낫다는 결과도 없습니다. 이 toy에서는 절대 flow가 모든 S에서 잔차 flow보다 높습니다. 기준 행동 a_0가 한 번의 Jacobian 역변환이라 잔차 분포가 더 복잡하고 생성기가 그 복잡도를 다 잡지 못한 것으로 보입니다.
- 따라서 "잔차 flow가 검색보다 낫다"는 조건은 이 toy에서 확인되지 않았습니다. 논문의 조건(HOT3D, 시연 수, 고차원 행동)과 다릅니다.

한계. 과제가 쉽습니다(절대 flow가 S = 256에서 0.999). 실패율 적합 a는 포화된 곡선에서 의미가 약합니다. 해가 곡선이라 "여러 해 중 하나를 맞추면 된다"는 설정이 실물 접촉보다 훨씬 관대합니다. U-Net, 학습 쌍 수, 기준 행동 품질을 바꾸면 순위가 달라질 수 있습니다. 시드 2개입니다.

추가 검증 (2026-11-14): 테이블 충돌 제약이 있는 평면 팔에서 잔차 flow, 절대 flow, 검색 비교

목적. 추가 검증 3은 자세 목표만 있는 과제라 검색이 거의 완벽했습니다. 이번에는 접촉 제약을 넣어 기준 행동 a_0가 성공 조건을 자주 어기도록 만들고, 같은 세 생성기를 두 학습 쌍 수에서 다시 비교합니다. 목표는 "잔차 flow가 검색보다 나은 조건"이 학습 쌍 밀도와 제약 때문에 달라지는지 보는 것입니다.

설정(가정).

- 과제: 추가 검증 3과 같은 3관절 평면 팔(링크 0.5, 0.35, 0.25 m). 성공은 (i) 끝점이 목표 c에서 2 cm 안이고 (ii) 모든 관절 점(팔꿈치, 손목, 끝점)의 y가 테이블 높이 −0.15 m 이상일 때입니다. 시뮬레이터 판정은 정확한 f와 이 제약으로 합니다.
- 이것은 접촉력이 아니라 **테이블 충돌(침투) 제약**을 넣은 toy입니다. 접촉 모델, 마찰, 힘 상한은 없습니다. 접촉 과제라고 부르기에는 단순화가 큽니다.
- 기준 행동 $a_0(c)$는 추가 검증 3과 같은 명목 자세 Jacobian 역변환 한 번입니다. 테이블 제약은 고려하지 않습니다.
- 학습 쌍: 관절각을 −2-2 rad에서 뽑아 성공 조건(테이블 위)을 만족하는 자세만 남깁니다(거부 표집). 학습 쌍 수 $N \in \{2{,}000, 40{,}000\}$.
- 시험 목표 500개는 테이블 위 자세의 끝점에서 뽑아, 성공 자세가 존재함을 보장합니다.
- MLP 256폭 3층(1D U-Net 대신, 추가 검증 3과 같은 이유), 3000 step, Euler 16 step, 시드 3개(0, 1, 2). 장치: CPU, torch 2.15.0, numpy 1.26.4, Python 3.12.7. 실행 77초.
- 생성기 세 개(잔차 flow, 절대 flow, 검색)와 pass@S, 실패율 적합 a는 추가 검증 3과 같습니다.

기준 행동의 상태. 시험 목표에서 a_0가 테이블 위에 있는 비율은 N과 무관하게 0.34-0.35입니다. 기준 행동 하나만 쓰면(S = 1, 성공 판정 포함) 성공률은 0.024입니다. 테이블 제약 때문에 IK 한 번으로는 거의 풀리지 않습니다.

결과 (pass@S, 시드 3개 평균 ± 표준편차).

N	생성기	S=1	S=4	S=16	S=64	S=256	실패율 적합 a
2,000	잔차 flow	0.34 ± 0.07	0.73 ± 0.06	0.93 ± 0.02	1.00 ± 0.00	1.00 ± 0.00	1.31 ± 0.06
2,000	절대 flow	0.49 ± 0.08	0.87 ± 0.07	0.98 ± 0.03	1.00 ± 0.01	1.00 ± 0.00	1.26 ± 0.02
2,000	검색	0.59 ± 0.02	0.74 ± 0.01	0.79 ± 0.01	0.81 ± 0.00	0.82 ± 0.01	0.14 ± 0.01 (포화)
40,000	잔차 flow	0.40 ± 0.02	0.75 ± 0.00	0.93 ± 0.02	0.99 ± 0.01	1.00 ± 0.01	1.09 ± 0.28
40,000	절대 flow	0.44 ± 0.14	0.82 ± 0.12	0.98 ± 0.02	1.00 ± 0.00	1.00 ± 0.00	1.33 ± 0.15
40,000	검색	0.96 ± 0.01	0.99 ± 0.01	0.99 ± 0.00	1.00 ± 0.00	1.00 ± 0.00	0.43 ± 0.04

결론.

- 학습 쌍이 적을 때(N = 2,000) 생성 모델이 검색을 넘습니다. S ≥ 4에서 flow가 검색(0.74-0.82에서 포화)보다 높고, S = 16에서 절대 flow 0.98, 잔차 flow 0.93, 검색 0.79입니다. 검색은 최근접 이웃의 잔차를 옮기는데, 이웃의 잔차가 새 목표의 테이블 제약을 자주 깨서 천장(약 0.82)에 머뭅니다.
- 학습 쌍이 많을 때(N = 40,000) 검색이 S = 1에서 0.96으로 flow(0.40-0.44)를 크게 이깁니다. 쌍이 촘촘하면 최근접 이웃이 목표 근처에서 제약을 만족하는 잔차를 그대로 줍니다. 추가 검증 3과 같은 방향입니다.
- 따라서 "flow가 검색보다 낫다"는 조건은 **학습 쌍이 희소하고 검색 천장이 낮을 때**로 좁아졌습니다. 제약 때문에 검색 천장이 생긴 것이 이 결과의 핵심입니다. 다만 천장은 N = 2,000과 toy 설정에 묶인 값입니다.
- 잔차 parameterization은 이득이 없습니다. 두 N 모두 S = 1에서 절대 flow가 잔차 flow보다 높거나 같습니다(N = 2,000에서 0.49 대 0.34). 기준 행동이 테이블을 뚫는 경우가 65%라서 잔차가 "조금 고치는" 구조가 되지 못하고, 잔차 분포가 오히려 복잡해진 것으로 보입니다.
- 실패율 적합 a는 이 toy에서 믿을 수 없습니다. flow 곡선은 S = 64 근처에서 이미 1.00에 도달해 적합 점이 포화 경계에 몰리고, 시드 간 sd가 큽니다(잔차 flow N = 40,000에서 1.09 ± 0.28). 검색 a는 포화된 곡선에서 나온 값이라 해석하지 않습니다. 추가 검증 1-3에서 본 "a ≈ 0.1-0.5" 패턴은 이 설정에서 재현되지 않았습니다.

한계. 접촉력이 없는 자세 제약 toy입니다. 해가 곡선이라 "여러 해 중 하나"만 맞추면 되는 설정입니다. MLP 한 종류, 학습 step 3000, 시드 3개입니다. 1D U-Net은 쓰지 않았습니다. 천장 값과 검색-생성 교차점은 제약 강도(테이블 높이)와 링크 수에 따라 달라질 수 있습니다. 논문의 접촉 과제(손가락 힘, 침투 SDF 판정)와는 다릅니다.

04. 주장 검증: 수치 재계산, 재현 정보, 구현 가능성

- **대상:** arXiv:2610.12440 v1(2026-10-08, 40쪽), 프로젝트 페이지(2026-11-03 저장, 저장소 `YY-GX/GNR` @ `d6cf09e`)
- **코드:** 공식 코드, 가중치, 데이터가 공개되지 않아(2026-11-03 확인) 코드 대조는 하지 않았습니다. 본문·표·부록 수치를 서로 대조해 다시 계산했습니다. 프로젝트 페이지 수치는 논문과 같습니다(README: "Numbers mirror the paper verbatim").
- **스크립트:** `analysis/table_check.py` (C1-C12), `analysis/scaling_toy.py` (pass@S·조기 종료), `analysis/manifold_toy.py` (가설 toy). 모두 CPU 단일 스레드이고 수 초에서 수십 초 걸립니다.

1. 수치 재계산

#	주장 (출처)	재계산	판정
C1	MPC 표본의 8.5%로 56.20% 대 27.20%, 궤적당 CPU 비용 7.6배 감소 (Abstract, Sec. 5.2)	256/3,000 = 8.53%. 2,689.3/354.1 = 7.59배(Table 1 단가)	산술 ✅. 27.20%는 C4
C2	Table 1 전체 SR (Table 1)	전체 = 0.272 × MPC-success + 0.728 × MPC-failure가 9개 행 모두 맞고, 성공 수가 정수(GNR S=256 = 252 + 310)	✅
C3	OOD 부분집합, MPC-success 평균 46.2 → 93.2%, MPC-failure 31.9-58.4% (Fig. 3, App. D.3)	부분집합별 MPC 성공 수 역산 37·69·166 = 272. 가중 평균이 Table 1의 S=1 18.1%, S=256 56.2%와 일치. MPC-success 평균 93.17%	✅. 부분집합 난이도 차이가 큼(MPC 성공 18·20·36%)
C4	DIAL-MPC 27.20% (Table 1)	272/1,000 = 시험 집합의 MPC-success 비율 자체. 행의 100%/0%는 정의. 모집단 비율 26.47%(Table 8)	⚠️ 비교 결과가 아니라 분할 레이블. 프로젝트 페이지 각주에만 설명 있음
C5	223k 시연, MPC-failure 127,136개 해결 (Sec. 5.2, App. C.5, D.4, Table 8)	기하 변형 쪽 시연 = 쌍 7,280 × 시드 24 = 174,720(평가 173,840). 쌍 단위 MPC 성공 37.4%, 시도 단위 27.6%. MPC-failure 중 17,434개(13.8%)는 MPC가 다른 시드로 푼 쌍. 본문 222,806 / 2,756 / 26.43% 대 Table 8 222,480 / 2,755 / 26.47%	⚠️ 시도 단위 계산. 본문·표 불일치(작음)
C6	실제 배치 2.62배 감소 (Sec. 5.2, App. G.2)	390.043 / 148.735 = 2.62 ✅. 단, MPC 시드는 Table 14 단가, GNR은 Table 1 단가. Table 14로 통일하면 2.56배. Table 14/Table 1 단가 비는 1.11-1.52. DIAL/GNR 비는 Table 14 기준 9.85배. 확장 비용은 조기 종료를 무시한 상한(App. G.2에 명시). 조기 종료 반영 추정 2.90배(`03` §2.3)	✅ 결론 유지. 단가 출처 혼용
C7a	ShapeFilter Table 10	DIAL-MPC 32×6(48.4)·CEM·iCEM·CMA-ES 값은 n=64에서 정수 비율. DIAL-MPC 500×6(92.9)은 589에서만. GNR 행(87.23, 92.26, 95.44, 97.37)은 64도 589도 아님	⚠️ 한 표 안에 분모가 섞임
C7b	S=1에서 52.4% (Sec. 5.3)	Table 11 State 50.42%, Table 12 N=2,448에서 역산 52.56%	⚠️ 같은 설정의 값이 표마다 다름
C7c	14.9배, 11.2배 (Table 2, App. G.2)	10,124/678 = 14.93. 8,541.0/765.6 = 11.16 ✅. Table 2의 GNR(77표본, 94.7%)은 2,448개 모델이 아니라 N=50 모델. MPC 2,500표본 93.7%(Table 2)와 3,000표본 92.9%(Table 10)가 따로 있음	✅ 산술. 비교 대상 설정 확인 필요
C7d	S=64에서 MPC 3,000표본 CPU 비용의 1.6% (Sec. 5.3)	Table 12는 조기 종료 비용. GNR 표본 단가 역산 38.6 core-s(MPC 롤아웃의 약 10배). S=64는 조기 종료 시 약 1.8%, 64개를 다 쓰면 약 21%	✅ 조기 종료 조건에서만
C8	NIST: GNR+MPC 92.2%를 33.3% 적은 표본으로, 3.0배, 1.8배 (Sec. 5.3, Table 2, App. F, G)	(3,440 − 2,296)/3,440 = 33.26% ✅. Table 2의 GNR+MPC는 1,976표본(42.6% 적음), 89.1% = 57/64, MPC 92.2% = 59/64. Fisher p = 0.76. Wilson [79.1, 94.6] 대 [83.0, 96.6]. 795,777/266,199 = 2.99 ✅. 79,136/44,517 = 1.78 ✅. 데이터셋 성공 (100 + 258 × 0.891)/358 = 92.12% ✅	✅ 산술. "비슷한 SR"은 표본 64개 수준의 비교
C9	다중 embodiment (Table 16)	Wilson 구간 4개 모두 재현. GNR S=384 95/100 대 SPIDER 96/100, Fisher p = 1.0	✅. GNR이 SPIDER를 넘지는 않음. 표본 수 단위도 다름(시연당 대 최적화당)
C10	BC 정책 37.5% 대 43.75% (App. I.1)	12/32 대 14/32, Fisher p = 0.80. 체크포인트 10개 중 같은 시험 32회로 고른 최고값	"쓸 수 있다"는 ✅, 우위 주장 근거는 없음

#	주장 (출처)	재계산	판정
C11	flywheel 55.2 → 74.4%, 68.4 → 84.8% (App. I.2)	시험 256개. 68.4·84.8은 정수 k/256으로 반올림 일치, 55.2·74.4는 불가(가장 가까운 값 55.08/55.47, 74.22/74.61)	⚠️ 작은 불일치. 평균값이거나 다른 시험 수일 가능성
C12	성공 기준과 "정밀도" (Table 3, Table 4, App. D.1, E.1)	HOT3D 위치 150 mm(CHORD 관례). MPC-failure 쪽 E_pos 중앙값 26-55 mm라서 위치 조건은 거의 걸러 내지 않음. ShapeFilter "1.96 mm"는 구멍 여유이고 판정은 부품 중심이 16 cm 삽입 영역 안인지. NIST 1 mm는 실제 판정 기준	⚠️ Table 3의 Precision 열은 성격이 다른 기준을 함께 놓음

추가 대조:

- **초록·서론의 "42.6%"**는 Table 1 MPC-failure 42.58%(310/728)입니다 ✅.
- **D.4의 360k 전체 58.94%** = (85,143 + 127,136)/360,150 ✅.
- **App. G.2의 시드 시도 수** 28,480/0.2643 = 107,756(논문 107,740) ✅. 2.62배의 분모 148.735k = 116.112k + 32.623k ✅.

2. 재현 정보 체크리스트

항목	공개 여부	비고
GNR 학습·추론 코드	✗	프로젝트 페이지에 "until release"
학습된 가중치	✗	
223k HOT3D 리타겟 데이터셋, 접촉력 라벨	✗	
ShapeFilter·NIST mocap 시연	✗	OptiTrack 15대 + 외골격
데이터 엔진(real-to-sim, 분할, 파지 재설정, 증강)	✗	절차는 App. C에 문장으로 있음
시뮬레이터 SuperDex	✓	Apache-2.0, FR3 + DG-5F 모델 포함, macOS(ARM) 지원 표기
DIAL-MPC	△	원 구현은 JAX + Brax(MuJoCo). SuperDex 위 구현은 비공개. 하이퍼파라미터는 Table 5에 있음
HOT3D	✓	projectaria.com 배포(약관 확인 필요)
SAM 3D, Point Transformer V3	✓	SAM License, CC BY-NC 4.0
모델 구조	△	U-Net 폭, 임베딩 차원, Euler 10단계(B.1). HOT3D의 에폭·학습률·잡음 일정은 없음
성공 기준 Γ	✓	Table 4
접촉 모델	△	강성 "기본값의 1/10", median filter 길이 미기재
기하 변형 200개/물체	△	종류와 개수(Table 7)는 있음. SAM 3D 입력 이미지, 변형 크기는 없음
기준선 설정	✓	Table 5, 6
시험 분할 목록	✗	1,000개 ID 없음

3. 프로젝트 페이지와 논문

프로젝트 페이지의 수치는 논문과 같습니다. 차이는 Table 1 각주 하나입니다. 페이지는 "DIAL-MPC generates GNR's training data and defines the MPC-success and MPC-failure splits, so its split success rates are 100% and 0% by construction"이라고 적었고, 논문에는 이 설명이 없습니다(C4). 저장소 마지막 커밋 메시지는 "Clarify GNR manifold and scaling copy and remove coming-soon links" (2026-10-30)입니다. 코드·데이터 버튼은 공개 전까지 숨겨 두었습니다.

4. 구현 가능성 판단

결론: 논문 결과는 **그대로 재현할 수 없습니다**. GNR 코드, 가중치, 데이터, 데이터 엔진이 모두 비공개입니다. 시뮬레이터(SuperDex)와 로봇 모델(FR3 + DG-5F), HOT3D, DIAL-MPC 원 구현은 공개되어 있습니다. 그래서 **방법을 작은 규모로 다시 구현하는 것은 가능**합니다. 어려운 부분은 생성 모델이 아니라 시드 데이터를 만드는 물리 리타겟팅 파이프라인입니다. 이 맥에서는 재계산과 toy를 끝냈습니다. SuperDex와 torch(MPS)를 설치하면 작은 규모의 재구현을 해 볼 수 있습니다(둘 다 설치 승인 필요).

4.1 논문 그대로 재현

불가능합니다.

- 코드·가중치·데이터·mocap 시연이 없습니다.
- 데이터 엔진(HOT3D → SuperDex 장면, keypoint IK, 파지 재설정, SuperDex 위 DIAL-MPC, Γ 판정, 증강)을 처음부터 다시 만들어야 합니다.
- 계산량이 큽니다. 전부 MPC로 돌리면 390k CPU core-h(512코어 31.7일)이고, 실제 배치 시나리오도 148.7k core-h입니다. HOT3D 모델 학습은 A100 1장입니다(시간 미보고).

4.2 이 맥에서 할 수 있는 것 (Apple M4 Pro, 통합 메모리 약 24 GiB, MPS, CUDA 없음)

작업	가능 여부	비고
표·부록 수치 재계산, pass@S 이질성 적합, manifold toy	**완료**	`table_check.py`, `scaling_toy.py`, `manifold_toy.py`
SuperDex 설치, FR3 + DG-5F 모델 로드, 궤적 재생	조건부	`uv pip install superdex`. macOS(ARM)가 지원 OS로 적혀 있지만 이 기기에서 시험하지 않았음. 설치 승인 필요
SuperDex 위 numpy annealed sampling MPC(DIAL-MPC 축소판), 단순 과제 하나	조건부	위 설치 뒤. CPU 롤아웃 속도는 미확인. 시연 수십 개 규모면 가능해 보임
작은 잔차 flow matching(1D temporal U-Net, 자세 조건) 학습	조건부	torch 설치(MPS). 19.57M 모델도 메모리상 가능. 데이터 수천 개 규모
Point Transformer V3 기하 인코더	조건부	2026-10부터 pip 패키지 `point-transformer-v3` 가 spconv·FlashAttention 없이 CPU·MPS에서 돈다고 README에 적혀 있음(미시험, CC BY-NC 4.0). 처음에는 자세 조건(ShapeFilter 본 실험 방식)으로 시작하는 편이 쉬움
HOT3D 다운로드와 전처리	조건부	약관 동의와 큰 다운로드 필요
논문 규모 재현	불가	수십만 core-h, A100·H100 학습

4.3 아이디어만 옮기는 경우

- **생성기 자체는 쉽습니다.** IK 기준에 대한 잔차 Δ를 조건부 flow matching으로 학습하고, Euler 10단계로 S개 뽑아 시뮬레이터에서 거르는 것은 수백 줄입니다.
- **비용의 대부분은 시드 생성기와 판정기입니다.** 접촉 많은 시뮬레이터에서 sampling MPC가 어느 정도 풀어야 하고(HOT3D 26%, 시드 시도당 1 core-h), Γ 같은 판정 기준을 신중하게 정해야 합니다. 판정이 느슨하면(C12) 성공률이 부풀려집니다.
- **먼저 할 기준선:** 최근접 시드 잔차 검색과 결정론적 회귀를 같은 거부 표집 틀에 넣어 비교해야 합니다. toy에서는 검색만으로도 범위 안에서 MPC를 넘었습니다(`03` §3).
- **보고 방식:** pass@S와 함께 조기 종료 기대 표본, 꼬리 지수 a, 쌍(문제) 단위 성공률을 같이 보고하는 편이 정확합니다(`03` §2, §4).

02. 선행연구와 계보

GNR은 물리 기반 리타겟팅의 "시연마다 최적화" 비용을, 풀린 궤적으로 학습한 조건부 생성기로 상각(amortize)하려는 연구입니다. 아래 서지는 논문 참고문헌에서 가져왔습니다. §4의 "논문이 인용하지 않은 연구"는 내 판단이고, 서지 일부는 원문을 다시 확인하지 않았습니다(해당 항목에 표시).

1. 계보도

```
[운동학 리타겟팅]  keypoint·손끝 IK
  DexPilot(Handa 2020) · AnyTeleop(Qin 2023) · GeoRT(Yin 2025, 신경망 IK)
        │  동역학 무시 → 물리에서 실행 불가 (HOT3D에서 IK 18.3%)
        ▼
[물리 기반 리타겟팅]
  ├─ RL: DexTrack(Liu 2025) · ManipTrans(Li 2025) · DexMachina(Mandi 2025) · CHORD(Zhu 2026)
  │        시연(또는 모음)마다 정책 학습, 보상 설계
  └─ sampling MPC: MPPI(Williams 2017) · iCEM(Pinneri 2021) · CMA-ES · DIAL-MPC(Xue 2024)
           → SPIDER(Pan 2025, annealed sampling + 접촉 유도) → Do as I Do(Paliwal 2026)
           시연마다 독립 최적화, 해가 다음 시연으로 옮겨 가지 않음
        │
        ▼  "풀린 궤적에서 배워 다음 시연을 싸게"
[상각]
  Generative Predictive Control(Kurtz & Burdick 2025): 과제별로 MPC를 flow matching 정책으로 증류
  GNR(2026): 시연(IK 레퍼런스 + 기하)을 조건으로 잔차 궤적을 생성
             open-loop pass@S  또는  MPC 초기값(GNR+MPC)
```

2. 구성 요소별 출처

구성 요소	출처	GNR에서의 역할
리타겟팅 정식화 (제약 최적화, 비용 J)	SPIDER (Pan et al., 2025)	Sec. 3.1의 문제 정의
시드 생성기, 기준선	DIAL-MPC (Xue et al., 2024)	HOT3D 300×10, ShapeFilter 500×6, NIST 3단계 (Table 5)
생성 모델	Flow matching (Lipman et al., 2022)	잔차 Δ의 조건부 생성, Euler 10단계
백본	Temporal U-Net (Ronneberger 2015), FiLM (Perez 2018)	19.57M 파라미터 (HOT3D)
기하 인코더	Point Transformer V3 (Wu et al., 2024)	점 1,024개
시뮬레이터	SuperDex Physics (The Mochi Team, 2026)	암시적 접촉, 분포 접촉력, 비볼록 충돌
사람 시연	HOT3D (Banerjee et al., 2025), OptiTrack 15대 + 착용형 외골격	다양성 축, 정밀도 축
기하 변형	SAM 3D (Chen et al., 2026b)	재생성, 가림 보완, 회전 후 재생성
성공 기준 150 mm	CHORD (Zhu et al., 2026)	HOT3D γ_track
정밀 과제	NIST assembly board (Lian et al., 2021)	M8 노브 체결
다중 embodiment 벤치마크	SPIDER 공개 벤치마크	손 5종, Table 16
pass@S	LLM 코드 생성의 pass@k 관례 (논문은 출처를 밝히지 않음)	성공 지표

3. 이 저장소의 다른 논문과의 관계

3.1 세 가지 리타겟팅 방식 비교

	운동학 (IK, GeoRT)	물리 기반 최적화 (RL, sampling MPC)	생성 (GNR)
출력	관절 자세 열	물리에서 검증된 제어열	제어열 후보 S개 (시뮬레이터로 거름)
시연 하나당 비용	ms 단위 (GNR 표의 IK 39 core-s는 시뮬레이터 재생 포함)	HOT3D DIAL-MPC 2,689 core-s, RL은 GPU-h 단위	HOT3D S=256에서 354 core-s, 별도 학습 비용
시연 사이 지식 전이	없음(규칙 고정)	MPC는 없음. 다중 레퍼런스 RL은 정책 안에서 공유	생성기가 공유
물리 실현 가능성	보장 없음	최적화 목표에 포함	생성 후 시뮬레이터 거부 표집으로 확보
전형적 실패	침투, 접촉 누락, 물체 낙하	국소해, 예산 부족, 보상 해킹	학습 분포 밖 외삽, 무거운 꼬리 (`03` §2-3)
필요한 것	손 모델	시뮬레이터, 비용·보상	시뮬레이터 + **시드 최적화기** + 학습 GPU

GNR은 물리 기반 최적화를 대체하지 않고 그 위에 얹습니다. 시드 데이터는 DIAL-MPC가 만들고, GNR은 그 해의 분포를 배워 제안 분포로 씁니다. 그래서 "생성 리타겟팅"보다 **학습된 제안 분포를 쓰는 sampling 리타겟팅**이라고 부르는 편이 정확합니다. 논문 제목의 "neural retargeting"은 GeoRT 같은 신경망 운동학 리타겟팅과 이름이 겹치지만, 출력이 물리 검증된 제어열이라는 점이 다릅니다.

3.2 개별 논문

논문 (폴더)	관계
FlashDexRetarget (`flashdexretarget/`)	목표가 같습니다. 둘 다 "시연마다 최적화"의 선형 비용을 없애려고 합니다. FDR은 레퍼런스 조건부 RL 정책 하나로 시연 모음 전체를 추적하고(폐루프, Isaac Sim, GPU-h), GNR은 MPC 해로 학습한 생성기에서 open-loop 후보를 뽑습니다(CPU core-h). 둘 다 HOT3D를 쓰지만, FDR의 엄격한 기준 SR_MT-Obj는 물체 3 cm / 30°이고 GNR HOT3D 기준은 150 mm / 40°입니다. 성공률을 직접 비교하면 안 됩니다. GNR은 FDR을 인용하지 않습니다(FDR v1 2026-10-01, GNR v1 2026-10-08 공개). FDR은 RL 학습 비용을 한 번 치르고, GNR은 MPC 시드 비용을 치릅니다. 두 방식 모두 "어떤 시연에서 실패하는가"는 비슷한 꼬리 문제를 보입니다(FDR 1,000개에서 변환 비율 61%, GNR MPC-failure 48%).
Do as I Do (`do_as_i_do/`)	GNR이 대체하려는 쪽입니다. SPIDER식 annealed sampling MPC(MuJoCo Warp, 1,024표본)로 단안 영상 복원 레퍼런스를 시연마다 따로 리타겟합니다. GNR Table 3에도 500개, 100 mm 기준으로 실렸습니다. Do as I Do의 transition reward처럼 비용을 고치는 방향과, GNR처럼 초기값·제안 분포를 고치는 방향은 함께 쓸 수 있습니다.
DexForce (`dexforce/`)	둘 다 "힘 라벨이 붙은 손 조작 데이터"를 만듭니다. DexForce는 손끝 6축 센서로 kinesthetic 시연의 **실측 힘**을 얻고(소규모), GNR은 시뮬레이터에서 **계산한 접촉력**을 대규모로 얻습니다. GNR 라벨은 접촉 강성(기본값의 1/10)에 의존하고 실측과 대조하지 않았습니다. DexForce의 x_f = x_o + k_f f 같은 힘 기반 행동을 GNR 데이터로 만들 수 있을지가 둘을 잇는 질문입니다.
Mana, **Lightning Grasp** (`mana/`, `lightning_grasp/`)	Zhao-Heng Yin 계열입니다. GNR이 운동학 리타겟팅의 예로 인용한 GeoRT의 저자이기도 합니다. Mana는 절차적 파지 keyframe 사이를 motion planning과 짧은 RL로 채우고, Lightning Grasp는 contact field로 파지를 빠르게 합성합니다. 둘 다 사람 시연 없이 파지를 만드는 쪽이고, GNR은 사람 시연을 물리로 옮기는 쪽입니다. GNR의 파지 재설정 단계(App. C.3)는 Lightning Grasp류 합성 파지로 바꿀 수 있는 자리입니다.
GraspSense (`graspsense/`)	둘 다 SAM 3D로 물체 메쉬를 만듭니다. GraspSense는 인식용이고, GNR은 같은 물체의 기하 변형(물체당 200개)을 만드는 데이터 증강에 씁니다. SAM 3D 메쉬의 물리 타당성을 검증하지 않는다는 점이 공통 약점입니다.
RoboSimStudio (`robosimstudio/`)	같은 SuperDex(`facebookresearch/project_superdex`)를 씁니다. RSS는 FEM 변형체용으로, GNR은 강체 손 조작 시뮬레이터로 씁니다.
Contact-Grounded Policy (`contact_grounded_policy/`)	같은 Meta Reality Labs Research입니다. CGP는 내부 FEM 시뮬레이터로 촉각 정책을 학습했습니다. GNR의 시뮬레이션 접촉력 라벨은 CGP 같은 접촉 기반 정책의 학습 데이터가 될 수 있습니다. 두 시뮬레이터가 같은 계열인지는 확인하지 못했습니다(가설).

4. 논문이 다루지 않았지만 관련 있는 연구 (내 판단)

- **FlashDexRetarget (arXiv:2610.01849):** §3.2. 같은 문제를 RL 쪽에서 상각합니다. 같은 HOT3D 클립과 같은 기준으로 비교하면 "생성기 + 거부 표집"과 "다중 레퍼런스 정책" 중 어느 쪽이 싼지 알 수 있습니다.
- **학습된 warm start로 궤적 최적화 가속:** Memory of Motion(Lembono et al., ICRA 2020)처럼, 풀린 문제에서 초기값 분포를 배워 최적화를 빠르게 하는 연구가 있습니다. GNR+MPC는 이 계열의 접촉 많은 손 조작 버전입니다. 논문은 이 계열을 인용하지 않습니다(서지는 다시 확인하지 않았음).
- **LLM의 test-time scaling과 검증기:** pass@k와 "많이 뽑고 실행해서 거르기"는 코드 생성에서 온 방식입니다(Codex, Chen et al., 2021). 무거운 꼬리 때문에 수익이 줄어드는 현상도 같은 구조입니다(`03` §2). 논문은 출처를 밝히지 않습니다.
- **잔차 학습:** IK 기준에 잔차를 더하는 설계는 FDR의 손목 잔차 행동, residual policy learning과 같은 발상입니다. App. E.4의 잔차 대 절대 비교는 이 선택을 뒷받침합니다.

5. GitHub 정리 (2026-11-03 `gh search repos`, `gh api` 확인)

공식 코드는 없습니다. 논문 제목, "GNR retargeting", 저자 계정(`YY-GX`, `gaodechen`)을 검색했고, 프로젝트 페이지 링크와 논문 본문에도 코드 주소는 없습니다.

저장소	무엇인가	공식 여부	라이선스	★	마지막 push	확인 커밋
YY-GX/GNR	프로젝트 페이지 정적 사이트(영상, 그림, 스크립트). 코드·데이터 없음. README: "Code and Dataset buttons are omitted until release"	공식(2저자 계정)	표기 없음	0	2026-10-30	`d6cf09e` (README와 파일 목록)
facebookresearch/project_superdex	SuperDex 엔진·Studio·Teleop. `assets/bots/arm_hand_combos/fr3_dg5f_*`에 FR3 + DG-5F 모델이 있음. 지원 OS에 macOS(ARM) 표기	논문이 쓴 시뮬레이터(GNR 코드 아님)	Apache-2.0	807	2026-11-03	`156fe13` (README와 파일 목록만)
LeCAR-Lab/dial-mpc	DIAL-MPC 원 구현. JAX + Brax(MuJoCo) 기반, CUDA 권장	시드 생성기 원본(GNR은 SuperDex 위에 다시 구현한 것으로 보임)	Apache-2.0	1,012	2025-05-28	`871c84f` (README만)
facebookresearch/spider	SPIDER 물리 기반 리타겟팅, 다중 embodiment 벤치마크	비교 기준선	CC BY-NC 4.0 (LICENSE 파일)	592	2026-09-22	`4471700` (메타데이터만)
facebookresearch/hot3d	HOT3D 데이터 API. 데이터는 projectaria.com·Hugging Face 배포(약관 확인 필요)	데이터	Apache-2.0 (코드)	292	2026-06-16	`146b34a` (메타데이터만)
Pointcept/PointTransformerV3	기하 인코더	구성 요소	CC BY-NC 4.0 (LICENSE 파일)	1,986	2026-10-31	`4e277e6` (메타데이터만)
facebookresearch/sam-3d-objects	기하 변형 생성	구성 요소	SAM License	7,626	2026-06-02	`f91db41` (메타데이터만)

clone한 저장소는 없습니다. GNR 코드가 없어서 코드 대조(04_code_verification) 대신 주장 검증(`04_claim_verification.md`)을 했습니다.